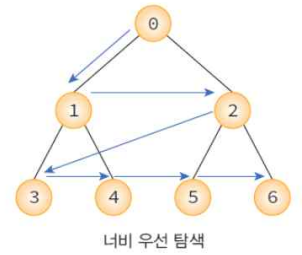
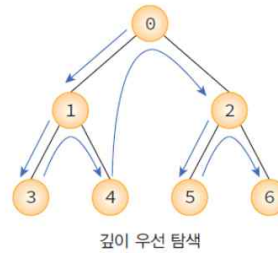


그래프 탐색의 종류

깊이 우선 탐색 : **DFS** Depth First Search
너비 우선 탐색 : **BFS** Breadth First Search



깊이 우선 탐색 DFS

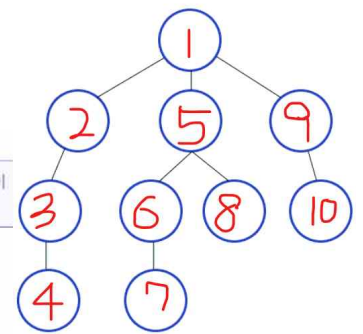
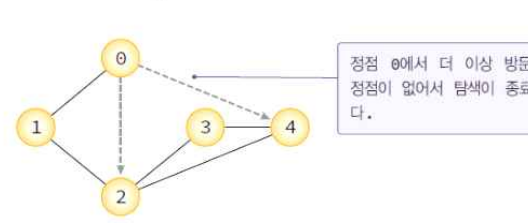
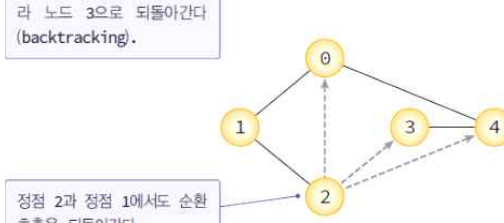
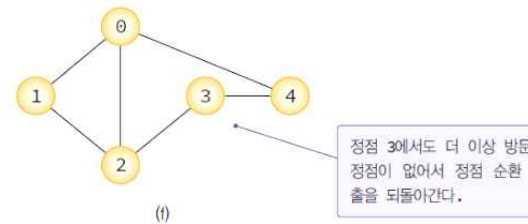
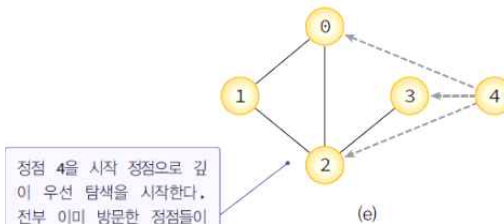
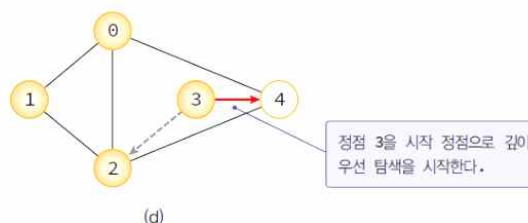
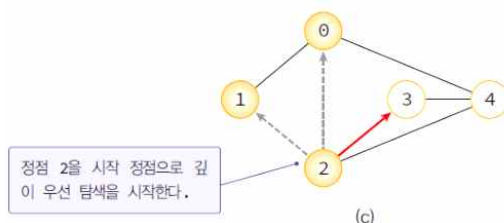
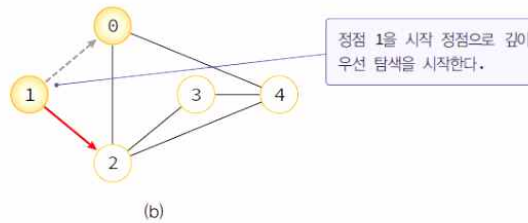
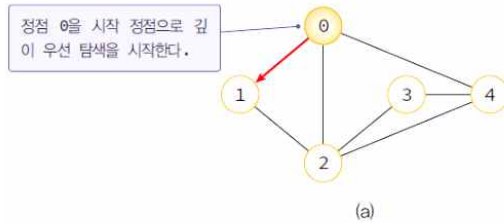
한 방향으로 갈 수 있을 때까지 가다가 더 이상 갈 수 없게 되면 가장 가까운 갈림길로 되돌아와서 그 곳에서부터 다른 방향으로 다시 탐색 진행.

되돌아가기 위해서는 **스택 필요**

```
// 정점 v 를 시작으로 깊이 우선 탐색
DFS(v)
    visited[v] ← TRUE;           //v는 이제 방문했다고 표시
    for u ∈ adjacent (v)         //v의 모든 인접 정점 u에 대해
        if (visited[u] == FALSE) //u를 아직 방문 안했으면
            DFS(u)                //u를 다시 DFS 탐색
```

방문 했는지 안했는지 flag로 표시
통상 제일 숫자가 낮은 것 선정.

가장 가까운 분기점까지 **backtracking!** > 그 분기점에서 탐색 안했던 곳으로 탐색



DFS의 구현2 : 인접 리스트 방식

DFS의 구현1 : 인접행렬 방식

// 인접 행렬로 표현된 그래프에 대한 깊이 우선 탐색
int visited[MAX]; //방문한 노드 표시

```
typedef struct GraphType{
    int n; //정점의 개수
    int adj_mat[MAX][MAX];
}GType;
```

0	1	1	0	0
1	1	1	1	0
2	1	1	0	1
3	0	1	0	0
4	0	0	1	0

void dfs_matrix(GType* g, int v)

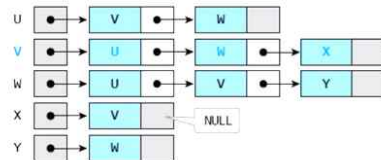
```
{
    int w;
    visited[v] = TRUE; // 정점 v의 방문 표시
    printf("정점 %d -> ", v); // 방문한 정점 출력
    for (w = 0; w < g->n; w++) // 인접 정점 탐색
        if (g->adj_mat[v][w] && !visited[w]) // 간선이 존재하고 && 아직 방문하지 않았다면
            dfs_matrix(g, w); // 정점 w에서 DFS 새로 시작 //재귀적으로 DFS
}
```

복잡도: $O(n^2)$

DFS의 구현2 : 인접리스트 방식

// 인접 리스트로 표현된 그래프에 대한 깊이 우선 탐색

```
int visited[MAX];
typedef struct GraphNode{
    int id;
    struct GraphNode* link;
}GNode;
typedef struct GraphType{
    int n; //정점의 개수
    GNode* adj_list[MAX];
}GType;
```



void dfs_list(GType* g, int v)

```
{
    GNode* w;
    visited[v] = TRUE; // 정점 v의 방문 표시
    printf("정점 %d -> ", v); // 방문한 정점 출력
    for (w = g->adj_list[v]; w != NULL; w = w->link) // 인접 정점 탐색
        if (!visited[w->id]) //방문한 적이 없으면...
            dfs_list(g, w->id); // 정점 w에서 DFS 새로 시작
}
```

복잡도: $O(n + e)$

너비 우선 탐색 BFS

시작 정점에서 가까운 정점을 먼저 방문하고 멀리 떨어져 있는 정점을 나중에 방문하는 순회 방법

'나'를 기준으로 같은 거리에 있는 것 우선 모두 파악

큐를 사용하여 구현 >> level order traversal과 유사!

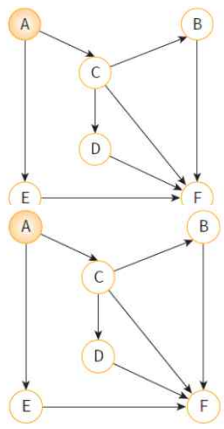
알고리즘

큐에서 정점을 꺼내어 정점을 방문하고 인접 정점들을 큐에 추가

큐가 소진될 때까지 동일한 코드 반복

// 정점 v 를 시작으로 깊이 우선 탐색

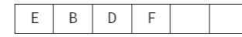
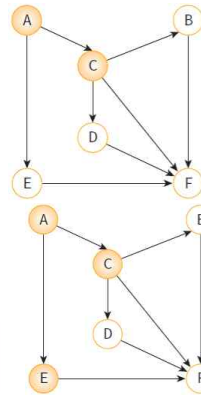
```
BFS(r)
    visited[r] ← TRUE; //v는 이제 방문했다고 표시
    enqueue(r); //v를 큐에 삽입
    while (!is_empty())
        v = dequeue();
        for u ∈ adjacent (v) //v의 모든 인접 정점 u에 대해
            if (visited[u] == FALSE) //u를 아직 미방문
                visited[u] ← TRUE //u를 방문했다고 표시
                enqueue(u)
```



처음에는 시작 정점인 A를 큐에 추가한다.



큐에서 A를 꺼내서 방문하고 정점 A의 인접 정점들을 큐에 추가한다.



큐에서 C를 꺼내서 방문하고 C의 인접 정점들을 큐에 추가한다.



큐에서 E를 꺼내서 방문하고 인접 정점인 F는 이미 큐에 들어 있으므로 추가하지 않는다.

이미 들어있으면 넣을 필요 X

BFS 알고리즘은 시작 정점으로부터 거리가 가까운 정점의 순서로 탐색을 진행
 거리가 **d**인 정점들을 모두 방문한 이후, 거리가 **(d+1)**인 정점들을 모두 방문한다
 이 과정을 **큐가 공백** 상태가 될 때까지 반복

BFS의 구현1 : 인접행렬 방식

// 인접 행렬로 표현된 그래프에 대한 너비 우선 탐색
 int visited[MAX];

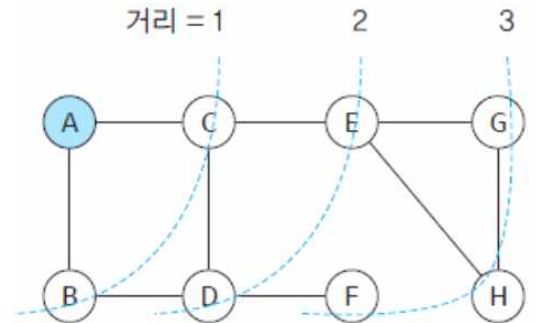
```
typedef struct GraphType{
    int n;          //정점의 개수
    int adj_mat[MAX][MAX];
}GType;
```

```
typedef struct QueueType{
    int queue[MAX_QUEUE];
    int front, rear;
}QType;
```

```
void bfs_matrix(GType* g, int v)
{
```

```
    int w;
    QType q;
    init_queue(&q);
    visited[v] = TRUE;           // 정점 v의 방문 표시
    printf("정점 %d -> ", v);   // 방문한 정점 출력
    enqueue(&q, v);             // 시작 정점을 큐에 저장
    while (!is_empty(&q)){
        v = dequeue(&q);        ///큐가 텅 비기 전까진
        for(w = 0; w < g->n; w++){ // 인접 정점 탐색
            if(g->adj_mat[v][w] && !visited[w]){
                visited[w] = TRUE; // 방문 표시
                printf("%d 방문 -> ", w);
                enqueue(&q, w); //방문한 정점을 큐에 저장
            }
        }
    }
}
```

복잡도: $O(n^2)$



BFS의 구현2 : 인접리스트 방식

// 인접 리스트로 표현된 그래프에 대한 너비 우선 탐색
 int visited[MAX];

```
typedef struct GNode{
    int id;
    struct GNode* link;
}GNode;
```

```
typedef struct GraphType{
    int n;          //정점의 개수
    GNode* adj_list[MAX];
}GType;
```

```

void bfs_list(GType* g, int v)
{
    GNode* w;
    QType q;
    init_queue(&q);
    visited[v] = TRUE;           // 정점 v의 방문 표시
    printf("정점 %d -> ", v);    // 방문한 정점 출력
    enqueue(&q, v);              // 시작 정점을 큐에 저장
    while (!is_empty(&q)){
        v = dequeue(&q);         // 큐에 정점 추출
        for(w = g->adj_list[v]; w != NULL; w = w->link){ // 인접 정점 탐색
            if(!visited[w->id]) {
                visited[w->id] = TRUE; // 방문 표시
                printf("%d 방문 -> ", w->id);
                enqueue(&q, w->id); // 방문한 정점 큐에 저장
            }
        }
    }
}

```

복잡도: $O(n + e)$

Lab : BFS vs DFS 요약

□ BFS

탐색순서: 한 층씩 **넓게**

장점: **최단 경로**를 보장함 (간선의 가중치가 동일한 경우)

단점: 큐에 다음 레벨의 모든 노드를 저장하므로 **메모리 사용량이 많음**.

노드 수가 많을 경우 큐의 크기가 급격히 증가 (공간 복잡도가 높음)

사용 예: **최단경로 탐색문제** (GPS 길찾기, 네트워크 최단경로)

□ DFS

탐색순서: 한 방향으로 **깊게**

장점: 경로의 길이가 긴 경우 빠르게 목표에 도달할 수 있음.

한 경로를 끝까지 탐색하므로 전체 노드를 메모리에 유지할 필요가 없어 메모리 사용량이 적음.

단점: 먼저 찾는 경로가 **최적(최단)이라는 보장이 없음**.

사이클이 있는 그래프의 경우 **무한 루프에 빠질 위험성**이 있음 (스택 오버플로우 발생)

사용 예:

경로추적문제: 스택구조(재귀호출)를 활용

백트래킹문제: 모든 가능한 수를 탐색해야 하는 문제에 적합 (예: 퍼즐)

10장 그래프 2

신장트리 (spanning tree)

그래프 내의 **모든 정점을 포함**하고, **최소 간선**의 수를 가지는 트리
n개의 정점을 가지는 그래프의 신장 트리는 **n-1개의 간선**을 가짐

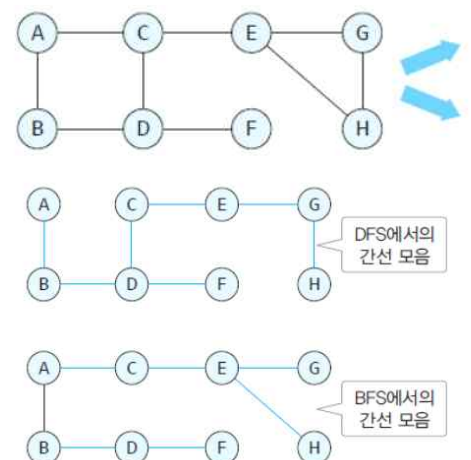
BFS나 DFS에서 사용한 간선들을 표시하기만 하면 됨

// DFS 알고리즘을 이용한 spanning tree
DFS(v)

```

visited[v] ← TRUE;           // v는 이제 방문했다고 표시
for u ∈ adjacent (v)         // v의 모든 인접 정점 u에 대해
    if (visited[u] == FALSE) // u를 아직 방문 안했으면
        Print edge (v, u)    // v, u 사이 간선을 표시
        DFS(u)                // u를 다시 DFS 탐색

```



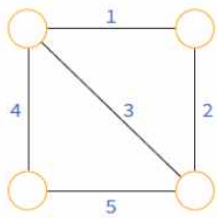
최소 신장 트리 Minimum Spanning Tree MST

트리를 구성하는 간선의 가중치 합이 최소인 신장트리

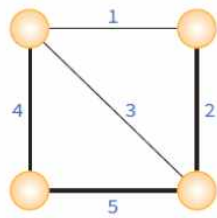
MST의 응용

도로 건설 : 도시들을 모두 연결하면서 도로의 길이를 최소화

통신 : 전화선의 길이가 최소가 되도록 전화 케이블망 구성

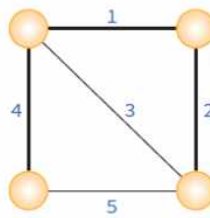


그래프



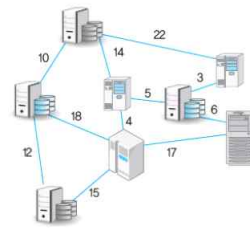
신장 트리

전체 비용=4+5+2=11

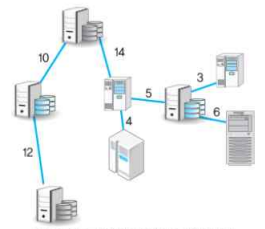


최소 비용 신장 트리

전체 비용=4+1+2=7



(a) 사이트 사이의 연결 비용



가중치 합 = 12+10+14+4+5+3+6=54

(b) 최소 비용의 통신망

모든 정점을 연결하고
비용이 최소인 트리가
MST이다.

Kruskal 알고리즘, Prim 알고리즘이 있다.

Kruskal 알고리즘

Greedy method 사용

매 순간 최선의 답을 선택하는 과정을 반복함으로써 최종적인 답에 도달하는 방법

알고리즘 설계시 많이 사용하는 기법

각 간선을 비용순으로 오름차순 정렬 후 최소 비용의 간선들을 차례로 선택

→ 각 간선 선택시 사이클 검사: union-find 알고리즘

각 단계별 local optimal 해답의 조합이 반드시 global optimal 해답이 된다는 보장이 없음

→ 최적의 해답을 주는지 검증이 필요함

Kruskal MST 알고리즘은 최적의 해답임이 증명됨

```
// 최소비용 스패닝트리를 구하는 Kruskal의 알고리즘
// 입력: 가중치 그래프  $G = (V, E)$ ,  $n$ 은 노드의 개수
// 출력:  $E_T$ , 최소비용 신장 트리를 이루는 간선들의 집합
kruskal(G)
```

E 를 $w(e_1) \leq \dots \leq w(e_n)$ 가 되도록 정렬한다.

$E_T \leftarrow \Phi$; $ecounter \leftarrow 0$

$k \leftarrow 0$

while $ecounter < (n - 1)$ do

$k \leftarrow k + 1$

if $E_T \cup \{e_k\}$ 가 사이클을 포함하지 않으면

then $E_T \leftarrow E_T \cup \{e_k\}$; $ecounter \leftarrow ecounter + 1$

return E_T

af	cd	bg	bc	dg	de	eg	ef	ab
10	12	15	16	18	22	25	27	29

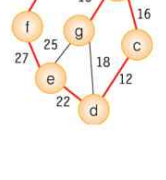
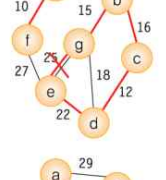
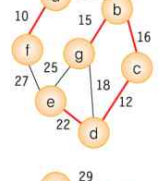
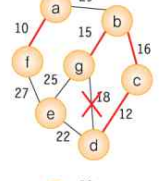
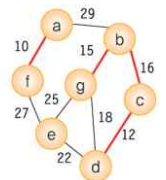
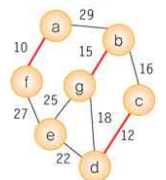
af	cd	bg	bc	dg	de	eg	ef	ab
10	12	15	16	18	22	25	27	29

af	cd	bg	bc	dg	de	eg	ef	ab
10	12	15	16	18	22	25	27	29

af	cd	bg	bc	dg	de	eg	ef	ab
10	12	15	16	18	22	25	27	29

af	cd	bg	bc	dg	de	eg	ef	ab
10	12	15	16	18	22	25	27	29

af	cd	bg	bc	dg	de	eg	ef	ab
10	12	15	16	18	22	25	27	29



순서대로 link를 하되,
cycle이 있으면 패스
-> union-find 알고리즘!

af	cd	bg	bc	dg	de	eg	ef	ab
10	12	15	16	18	22	25	27	29

af	cd	bg	bc	dg	de	eg	ef	ab
10	12	15	16	18	22	25	27	29

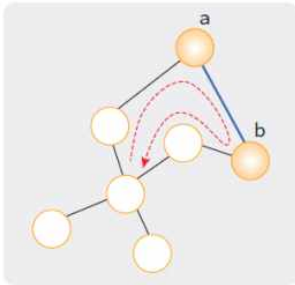
Union-find 알고리즘

원소가 어떤 집합에 속하는지 알아냄.

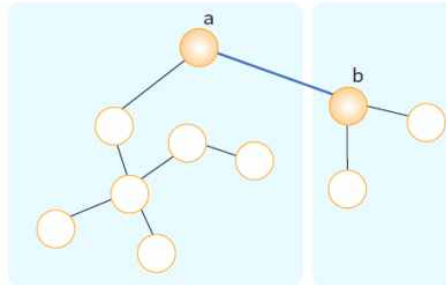
이 자체는 사이클을 찾는 알고리즘이 아니지만... Kruskal 알고리즘에서 사이클 검사에 사용

* 연결된 정점들은 같은 집합에 속한다고 가정.

* 추가할 간선의 두 정점이 같은 집합에 속하면 사이클이 발생.



(a) 사이클 형성



(b) 사이클 형성되지 않음

단계	그래프	최소 신장트리(MST)
0	간선들을 가중치의 오름차순으로 정렬 BC 10, AD 12, EG 14, BE 15, CG 16, DE 17, EF 19, AB 25, DF 37, FG 43	
1	최소 간선 (B,C)는 사이클을 만들지 않으므로 MST에 추가 BC 10, AD 12, EG 14, BE 15, CG 16, DE 17, EF 19, AB 25, DF 37, FG 43	
2	다음 간선 (A,D)도 사이클을 만들지 않으므로 MST에 추가 BC 10, AD 12, EG 14, BE 15, CG 16, DE 17, EF 19, AB 25, DF 37, FG 43	
3	다음 간선 (E,G)도 사이클을 만들지 않으므로 MST에 추가 BC 10, AD 12, EG 14, BE 15, CG 16, DE 17, EF 19, AB 25, DF 37, FG 43	
4	다음 간선 (B,E)도 사이클을 만들지 않으므로 MST에 추가 BC 10, AD 12, EG 14, BE 15, CG 16, DE 17, EF 19, AB 25, DF 37, FG 43	
5	간선 (C,G)는 사이클을 만들기 때문에 버림. 다음 간선으로 넘어감. BC 10, AD 12, EG 14, BE 15, CG 16 , DE 17, EF 19, AB 25, DF 37, FG 43	
6	다음 간선 (D,E)는 사이클을 만들지 않으므로 MST에 추가 BC 10, AD 12, EG 14, BE 15, CG 16 , DE 17, EF 19, AB 25, DF 37, FG 43	
7	(E,F)도 사이클을 만들지 않으므로 MST에 추가. 간선의 수가 n-1 → 종료. BC 10, AD 12, EG 14, BE 15, CG 16 , DE 17, EF 19, AB 25, DF 37, FG 43	

Kruskal의 복잡도

대부분 edge를 정렬하는 시간에 좌우됨. 나머지 계산은 신속.

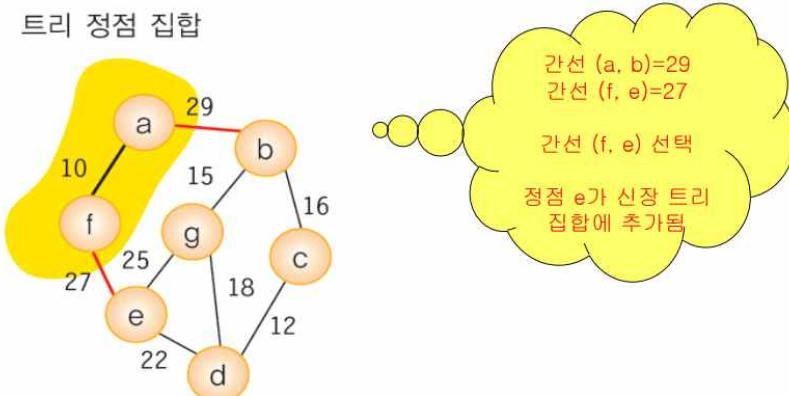
n개의 edge를 정렬할 때 쿼정렬로 하면 Kruskal 알고리즘의 시간복잡도는 $O(n \log n)$ 이 된다.

Prim 알고리즘

시작 정점에서부터 출발하여 spanning tree 집합을 단계적으로 확장하는 방법

인접한 정점들 중 최저 간선으로 연결된 정점을 선택하여 spanning tree 집합에 추가

트리 정점 집합



알고리즘 8.3

PRIM의 최소비용 신장트리

```

01 Prim(s)
02   MST ← { s }                                // 맨 처음에는 시작 정점만 포함
03   dist[s] ← 0                                // s까지의 거리는 0, 나머지는 ∞
04   for i←1 to vsize-1 :                        // 모든 정점이 포함될 때까지
05       v ← getMinVertex()                      // 간선의 가중치가 가장 작은 정점 선택
06       MST ← MST ∪ { v }                      // v를 MST에 추가
07       for w ∈ adjacent(v) :                  // v의 모든 인접 정점 w에 대해
08           if w ∉ MST :                        // w가 MST에 포함되지 않았으면
09               if cost(v,w) < dist[w] :        // (v,w)의 가중치가 dist[w]보다 작으면
10                   dist[w] ← cost(v,w)        // dist[w] 갱신
    
```

□ dist[n] 배열

- 신장트리 정점 집합에서 각 정점까지 현재까지 알려진 최소의 가중치 값을 저장
- dist[] 에서 최소 가중치값을 가지는 노드 v를 매 단계 선택
- 새로운 정점 v가 신장트리 집합에 추가되면 v의 인접정점 w들의 해당 dist[w]값들을 업데이트 (새로운 인접 가중치 값이 기존 dist[w]값보다 작을 경우)

Kruskal은 edge 중심, Prim은 노드 중심!

단계	그래프	최소 신장트리(MST)
0 맨 처음에 MST는 공백 상태이다. dist[]는 시작 정점 A만 0이고 나머지는 ∞로 초기화된다. MST = { }		
1 dist가 최소인 A를 MST에 넣고, A의 인접 정점들의 dist를 갱신한다. A의 인접 정점 B와 D의 거리가 25, 12로 변경된다. MST = { A }		

2	dist가 최소인 D를 선택해 MST에 넣는다. D의 이웃 정점 E와 F의 거리가 각각 17과 37로 갱신된다. MST = {A, D} <table> <tr> <th></th><th>A</th><th>B</th><th>C</th><th>D</th><th>E</th><th>F</th><th>G</th></tr> <tr> <td>dist</td><td>0</td><td>25</td><td>∞</td><td>12</td><td>17</td><td>37</td><td>∞</td></tr> </table>		A	B	C	D	E	F	G	dist	0	25	∞	12	17	37	∞		
	A	B	C	D	E	F	G												
dist	0	25	∞	12	17	37	∞												
3	dist가 최소인 E를 MST에 넣고, 이웃 정점 B, F, G의 거리가 각각 15, 19, 14로 갱신된다. MST = {A, D, E} <table> <tr> <th></th><th>A</th><th>B</th><th>C</th><th>D</th><th>E</th><th>F</th><th>G</th></tr> <tr> <td>dist</td><td>0</td><td>15</td><td>∞</td><td>12</td><td>17</td><td>19</td><td>14</td></tr> </table>		A	B	C	D	E	F	G	dist	0	15	∞	12	17	19	14		
	A	B	C	D	E	F	G												
dist	0	15	∞	12	17	19	14												
4	dist가 최소인 G가 선택되고, 인접 정점 C, F 중에서 C 까지의 거리만 16으로 갱신된다. MST = {A, D, E, G} <table> <tr> <th></th><th>A</th><th>B</th><th>C</th><th>D</th><th>E</th><th>F</th><th>G</th></tr> <tr> <td>dist</td><td>0</td><td>15</td><td>16</td><td>12</td><td>17</td><td>19</td><td>14</td></tr> </table>		A	B	C	D	E	F	G	dist	0	15	16	12	17	19	14		
	A	B	C	D	E	F	G												
dist	0	15	16	12	17	19	14												
5	dist가 최소인 B가 선택되고, 인접 정점 C의 거리가 10으로 갱신된다. MST = {A, D, E, G, B} <table> <tr> <th></th><th>A</th><th>B</th><th>C</th><th>D</th><th>E</th><th>F</th><th>G</th></tr> <tr> <td>dist</td><td>0</td><td>15</td><td>10</td><td>12</td><td>17</td><td>19</td><td>14</td></tr> </table>		A	B	C	D	E	F	G	dist	0	15	10	12	17	19	14		
	A	B	C	D	E	F	G												
dist	0	15	10	12	17	19	14												
6	dist가 최소인 C가 선택되고, 선택되지 않은 인접 정점은 없다. MST = {A, D, E, G, B, C} <table> <tr> <th></th><th>A</th><th>B</th><th>C</th><th>D</th><th>E</th><th>F</th><th>G</th></tr> <tr> <td>dist</td><td>0</td><td>15</td><td>10</td><td>12</td><td>17</td><td>19</td><td>14</td></tr> </table>		A	B	C	D	E	F	G	dist	0	15	10	12	17	19	14		
	A	B	C	D	E	F	G												
dist	0	15	10	12	17	19	14												
7	마지막으로 dist가 최소인 F를 추가하면 MST가 완성된다. MST = {A, D, E, G, B, C, F} <table> <tr> <th></th><th>A</th><th>B</th><th>C</th><th>D</th><th>E</th><th>F</th><th>G</th></tr> <tr> <td>dist</td><td>0</td><td>15</td><td>10</td><td>12</td><td>17</td><td>19</td><td>14</td></tr> </table>		A	B	C	D	E	F	G	dist	0	15	10	12	17	19	14		
	A	B	C	D	E	F	G												
dist	0	15	10	12	17	19	14												

Prim의 복잡도

주 반복문이 정점의 수 n 만큼 반복하고, 내부 반복문이 n 번 반복하므로 **Prim 알고리즘은 $O(n^2)$** 이다

Kruskal vs Prim

Kruskal : 간선(edge)를 기반으로 하는 알고리즘.

이전 단계에서 만들어진 신장트리와는 **무관**하게 최저 edge 값을 선택

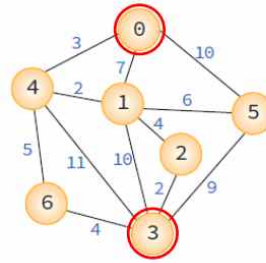
Prim : 정점(vertex)를 기반으로 하는 알고리즘.

이전 단계에서 만들어졌던 신장트리를 **확장**하는 방식

최단 경로

vertex u 와 vertex v 를 연결하는 경로 중에서 edge들의 **가중치 합이 최소**가 되는 경로
가중치는 비용, 거리, 시간 등

- Dijkstra 알고리즘
- Floyd 알고리즘



	0	1	2	3	4	4	5
0	0	7	∞	∞	3	10	∞
1	7	0	4	10	2	6	∞
2	∞	4	0	2	∞	∞	∞
3	∞	10	2	0	11	9	4
4	3	2	∞	11	0	∞	5
5	10	6	∞	9	∞	0	∞
6	∞	∞	∞	4	5	∞	0

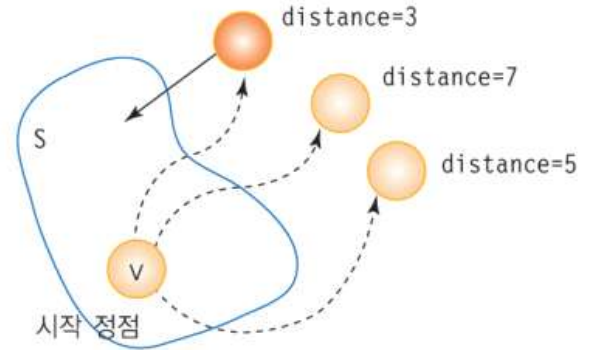
Dijkstra 알고리즘

하나의 시작 vertex로부터 모든 다른 vertex까지의 최단 경로 찾기

집합 S : 시작 정점 v 로부터의 최단경로가 이미 발견된 정점들의 집합

distance [] : 최단경로가 알려진 정점들만을 이용한 다른 정점들까지의 최단경로 길이(Prim 알고리즘)

매 단계에서 가장 작은 distance값을 가진 정점을 S 에 추가.

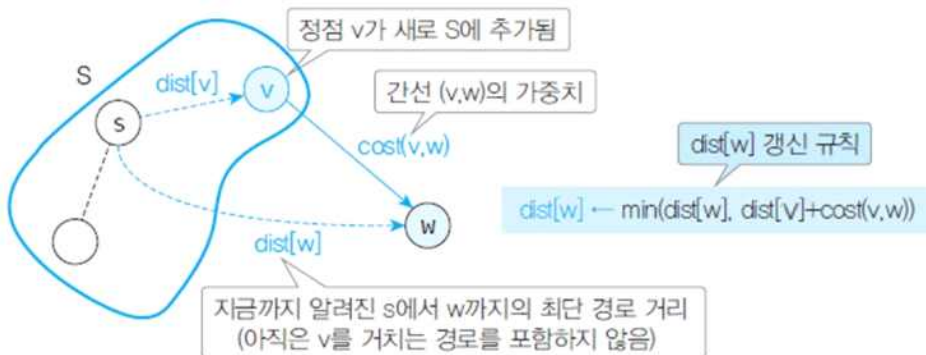


기본 아이디어

S 에 포함되지 않은 정점 중에 **dist가 최소인 정점 v** 를 찾음

v 까지의 거리를 확정하고 S 에 추가

새로운 정점 v 가 S 에 추가되면 **v 인접 정점의 거리 갱신**



$v + (v,w)$ 거리와 w 거리 중
작은 것으로 갱신!

알고리즘 8.4

Dijkstra의 최단 경로 거리

```

01 Dijkstra(s)
02   S ← { }                                // 확정된 정점 없음
03   for v ∈ G :                             // 모든 정점에 대해
04       dist[v] ← ∞                         // 모든 정점의 dist를 ∞로 초기화
05   dist[s] ← 0                             // 시작 정점까지의 거리만 0으로 초기화
06   for i=1 to vsize :                      // 모든 정점이 포함될 때까지
07       v ← getMinVertex()                  // 간선의 가중치가 가장 작은 정점 선택
08       S ← S ∪ {v}                        // v를 S에 추가
09       for w ∈ adjacent(v) :               // v의 모든 인접 정점 w에 대해
10           if w ∉ S :                      // w가 S에 포함되지 않았으면
11               if dist[v]+weight[v][w] < dist[w] : // dist갱신
12                   dist[w] ← dist[v]+weight[v][w];
    
```

단계	집합 S와 dist 변화	그래프														
0	맨 처음에 시작 정점만 0이고, 다른 정점들은 모두 ∞의 거리를 가짐. $S = \{ \}$ S포함 dist <table><tr><td>A</td><td>B</td><td>C</td><td>D</td><td>E</td><td>F</td><td>G</td></tr><tr><td>0</td><td>∞</td><td>∞</td><td>∞</td><td>∞</td><td>∞</td><td>∞</td></tr></table>	A	B	C	D	E	F	G	0	∞	∞	∞	∞	∞	∞	
A	B	C	D	E	F	G										
0	∞	∞	∞	∞	∞	∞										
1	dist가 최소인 A가 선택되고, A의 인접 정점 B와 D까지의 거리가 갱신됨. $S = \{ A \}$ S포함 dist <table><tr><td>A</td><td>B</td><td>C</td><td>D</td><td>E</td><td>F</td><td>G</td></tr><tr><td>0</td><td>25</td><td>∞</td><td>12</td><td>∞</td><td>∞</td><td>∞</td></tr></table>	A	B	C	D	E	F	G	0	25	∞	12	∞	∞	∞	
A	B	C	D	E	F	G										
0	25	∞	12	∞	∞	∞										
2	dist가 최소인 D가 선택되고, E와 F까지의 거리가 갱신됨. $S = \{ A, D \}$ S포함 dist <table><tr><td>A</td><td>B</td><td>C</td><td>D</td><td>E</td><td>F</td><td>G</td></tr><tr><td>0</td><td>25</td><td>∞</td><td>12</td><td>29</td><td>49</td><td>∞</td></tr></table>	A	B	C	D	E	F	G	0	25	∞	12	29	49	∞	
A	B	C	D	E	F	G										
0	25	∞	12	29	49	∞										
3	dist가 최소인 B가 선택되고, C와 E 중에서 C만 거리가 갱신됨. $S = \{ A, D, B \}$ S포함 dist <table><tr><td>A</td><td>B</td><td>C</td><td>D</td><td>E</td><td>F</td><td>G</td></tr><tr><td>0</td><td>25</td><td>35</td><td>12</td><td>29</td><td>49</td><td>∞</td></tr></table>	A	B	C	D	E	F	G	0	25	35	12	29	49	∞	
A	B	C	D	E	F	G										
0	25	35	12	29	49	∞										
4	dist가 최소인 E가 선택되고, F와 G의 거리가 모두 갱신됨. $S = \{ A, D, B, E \}$ S포함 dist <table><tr><td>A</td><td>B</td><td>C</td><td>D</td><td>E</td><td>F</td><td>G</td></tr><tr><td>0</td><td>25</td><td>35</td><td>12</td><td>29</td><td>48</td><td>43</td></tr></table>	A	B	C	D	E	F	G	0	25	35	12	29	48	43	
A	B	C	D	E	F	G										
0	25	35	12	29	48	43										
5	dist가 최소인 C가 선택되고, G의 거리는 갱신되지 않음. $S = \{ A, D, B, E, C \}$ S포함 dist <table><tr><td>A</td><td>B</td><td>C</td><td>D</td><td>E</td><td>F</td><td>G</td></tr><tr><td>0</td><td>25</td><td>35</td><td>12</td><td>29</td><td>48</td><td>43</td></tr></table>	A	B	C	D	E	F	G	0	25	35	12	29	48	43	
A	B	C	D	E	F	G										
0	25	35	12	29	48	43										
6	dist가 최소인 G가 선택되고, F의 거리는 갱신되지 않음. $S = \{ A, D, B, E, C, G \}$ S포함 dist <table><tr><td>A</td><td>B</td><td>C</td><td>D</td><td>E</td><td>F</td><td>G</td></tr><tr><td>0</td><td>25</td><td>35</td><td>12</td><td>29</td><td>48</td><td>43</td></tr></table>	A	B	C	D	E	F	G	0	25	35	12	29	48	43	
A	B	C	D	E	F	G										
0	25	35	12	29	48	43										
7	F가 마지막으로 선택. 모든 정점까지의 최단 경로 확정. $S = \{ A, D, B, E, C, G, F \}$ S포함 dist <table><tr><td>A</td><td>B</td><td>C</td><td>D</td><td>E</td><td>F</td><td>G</td></tr><tr><td>0</td><td>25</td><td>35</td><td>12</td><td>29</td><td>48</td><td>43</td></tr></table>	A	B	C	D	E	F	G	0	25	35	12	29	48	43	
A	B	C	D	E	F	G										
0	25	35	12	29	48	43										

Dijkstra 알고리즘 복잡도

주 반복분이 정점의 수 n 만큼 반복하고, 내부 반복문이 n 번 반복하므로 Dijkstra 알고리즘은 n^2) 복잡도를 가짐.

Floyd 알고리즘

모든 정점 사이의 최단 경로를 한꺼번에 찾음

최단경로 거리행렬 A

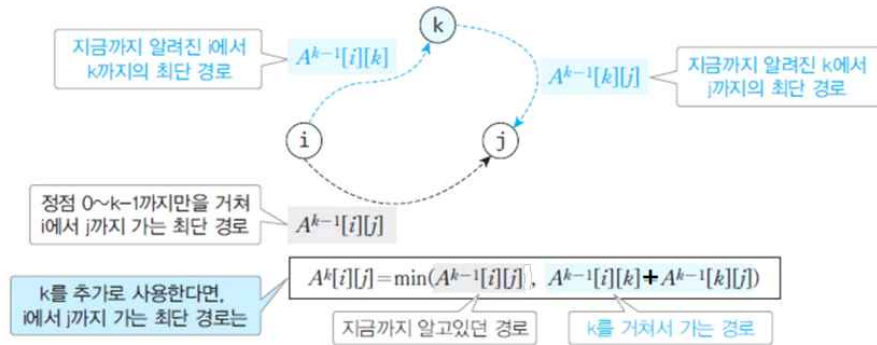
2차원 배열 A를 이용하여 3중 반복문으로 구성

$i[j]$ -> 0부터 k까지의 정점만을 이용한 정점 i에서 j까지의 최단 경로 길이

$A^{-1} \rightarrow A^0 \rightarrow A^1 \rightarrow \dots \rightarrow A^{n-1}$ 순으로 최단 경로 구함.

기본 아이디어

- A^{k-1} 까지 구해진 상태에서 k번째 정점이 추가로 고려되는 상황을 생각하자



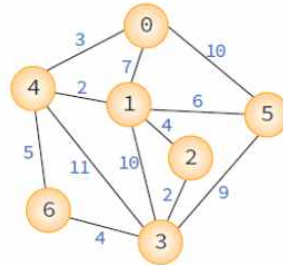
알고리즘 8.5 Floyd의 최단 경로 거리

```

01 Floyd()
02   for k=0 to vsize-1 :           // 각 정점을 순서대로 고려함
03     for i=0 to vsize-1 :         // 모든 행에 대해
04       for j=0 to vsize-1 :       // 모든 열에 대해
05         A[i][j] ← min(A[i][j], A[i][k]+A[k][j])
    
```

- (1) 그래프의 가중치 행렬로 배열 A를 초기화한다.

0	7	*	*	3	10	*
7	0	4	10	2	6	*
*	4	0	2	*	*	*
*	10	2	0	11	9	4
3	2	*	11	0	*	5
10	6	*	9	*	0	*
*	*	*	4	5	*	0



- (2) 정점 0을 거쳐서 가는 경로와 비교하여 최단 경로를 수정한다.

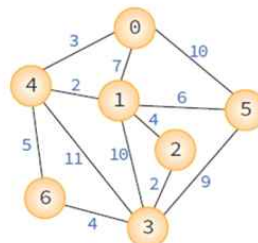
0	7	*	*	3	10	*
7	0	4	10	2	6	*
*	4	0	2	*	*	*
*	10	2	0	11	9	4
3	2	*	11	0	13	5
10	6	*	9	13	0	*
*	*	*	4	5	*	0

$$A^0[i][j] = \min(A^{-1}[i][j], A^{-1}[i][0] + A^{-1}[0][j])$$

- (3) 정점 1을 거쳐서 가는 경로와 비교하여 최단 경로를 수정한다.

0	7	11	17	3	10	*
7	0	4	10	2	6	*
11	4	0	2	6	10	*
17	10	2	0	11	9	4
3	2	6	11	0	8	5
10	6	10	9	8	0	*
*	*	*	4	5	*	0

$$A^1[i][j] = \min(A^0[i][j], A^0[i][1] + A^0[1][j])$$




```
typedef struct GraphType{
    int n; //정점의 개수
    int weight[MAX][MAX]; //인접행렬
}GType;

int A[MAX][MAX]; //최단경로거리행렬
```

```
void Floyd(GType* g)
```

```
{
    int i, j, k;
    for (i = 0; i < g->n; i++)
        for (j = 0; j < g->n; j++)
            A[i][j] = g->weight[i][j];
    printA(g);

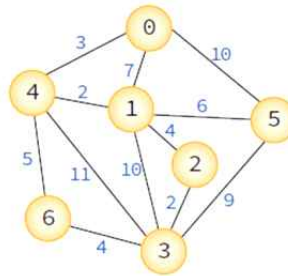
    for (k = 0; k < g->n; k++) {
        for (i = 0; i < g->n; i++)
            for (j = 0; j < g->n; j++)
                if (A[i][k] + A[k][j] < A[i][j])
                    A[i][j] = A[i][k] + A[k][j];
        printA(g);
    }
}
```

인접행렬을 최단경로
거리행렬 A에 복사

```
int main(void)
```

```
{
    GType g = { 7,
        {{ 0, 7, INF, INF, 3, 10, INF },
        { 7, 0, 4, 10, 2, 6, INF },
        { INF, 4, 0, 2, INF, INF, INF },
        { INF, 10, 2, 0, 11, 9, 4 },
        { 3, 2, INF, 11, 0, INF, 5 },
        { 10, 6, INF, 9, INF, 0, INF },
        { INF, INF, INF, 4, 5, INF, 0 } }
    };

    Floyd(&g);
    return 0;
}
```



	0	1	2	3	4	5	6
0	0	7	INF	INF	3	10	INF
1	7	0	4	10	2	6	INF
2	INF	4	0	2	INF	INF	INF
3	INF	10	2	0	11	9	4
4	3	2	INF	11	0	13	5
5	10	6	INF	9	13	0	INF
6	INF	INF	INF	4	5	INF	0

	0	1	2	3	4	5	6
0	0	7	11	17	3	10	INF
1	7	0	4	10	2	6	INF
2	11	4	0	2	6	10	INF
3	17	10	2	0	11	9	4
4	3	2	6	11	0	8	5
5	10	6	10	9	8	0	INF
6	INF	INF	INF	4	5	INF	0

	0	1	2	3	4	5	6
0	0	7	11	13	3	10	INF
1	7	0	4	6	2	6	INF
2	11	4	0	2	6	10	INF
3	13	6	2	0	8	9	4
4	3	2	6	8	0	8	5
5	10	6	10	9	8	0	INF
6	INF	INF	INF	4	5	INF	0

	0	1	2	3	4	5	6
0	0	7	11	13	3	10	17
1	7	0	4	6	2	6	10
2	11	4	0	2	6	10	6
3	13	6	2	0	8	9	4
4	3	2	6	8	0	8	5
5	10	6	10	9	8	0	3
6	17	10	6	4	5	13	0

	0	1	2	3	4	5	6
0	0	5	9	11	3	10	8
1	5	0	4	6	2	6	7
2	9	4	0	2	6	10	6
3	11	6	2	0	8	9	4
4	3	2	6	8	0	8	5
5	10	6	10	9	8	0	13
6	8	7	6	4	5	13	0

	0	1	2	3	4	5	6
0	0	5	9	11	3	10	8
1	5	0	4	6	2	6	7
2	9	4	0	2	6	10	6
3	11	6	2	0	8	9	4
4	3	2	6	8	0	8	5
5	10	6	10	9	8	0	13
6	8	7	6	4	5	13	10

	0	1	2	3	4	5	6
0	0	5	9	11	3	10	8
1	5	0	4	6	2	6	7
2	9	4	0	2	6	10	6
3	11	6	2	0	8	9	4
4	3	2	6	8	0	8	5
5	10	6	10	9	8	0	13
6	8	7	6	4	5	13	0

Floyd 알고리즘 복잡도

n개의 정점이 있다면, floyd의 최단 경로 알고리즘은 3중 반복문이 실행되므로 시간 복잡도는 $O(n^3)$ 이 된다.

모든 정점상의 최단경로를 구하려면 Dijkstra의 알고리즘 (n^2)을 n번 반복해도 되며, 이 경우 전체 복잡도는 $O(n^3)$ 이 된다.

11장 정렬 Sorting

정렬

사물들을 크기 순으로 오름차순이나 내림차순으로 나열하는 작업.
자료 탐색에 있어서 필수적.

일반적으로 정렬시켜야 될 대상은 레코드(record) ---- 레코드는 필드(field)들로 구성되어 있음.
키(key) 필드로 레코드를 구별함.

>> 정렬이란 key 순서로 재배열 하는 작업

정렬 알고리즘

모든 경우에 최적인 정렬 알고리즘은 없음. (universal한 것은 없음!)

>> 각 응용 분야에 적합한 정렬 방법 사용해야 함.

레코드 개수, 크기, key의 특성(문자, 정수, 실수 등)

정렬의 종류

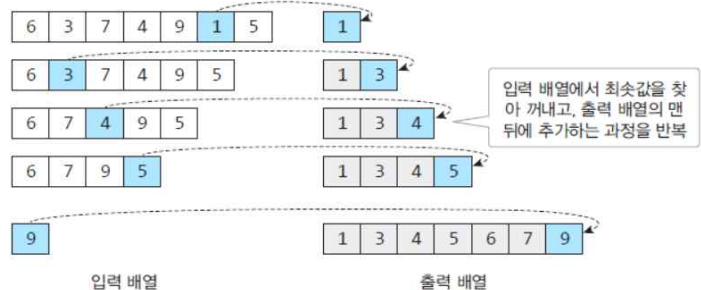
단순하지만 비효율적인 방법 : 삽입, 선택, 버블정렬

복잡하지만 효율적인 방법 : 퀵, 병합, 기수정렬 등

1. 선택 정렬 Selection Sort

가장 작은 숫자부터 하나씩 찾아 순서대로 저장

- 배열에서 가장 작은 값을 찾아 출력하고, 남은 자료 중 다시 가장 작은 값을 찾아서 출력하는 과정을 반복



- 최소값을 출력 배열에 저장하는 것이 아니라, 입력 배열의 첫번째 요소와 교환 → 추가 배열을 요구하지 않음

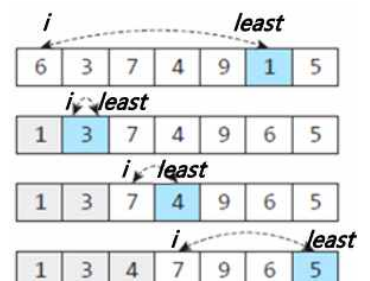
in-place sorting이 아님!

>> 개선해보자! **SWAP**하기



알고리즘

```
selection_sort(A, n)
for i ← 0 to n-1 do
    least ← A[i], A[i+1], ..., A[n-1] 중에서 가장 작은 값의 인덱스;
    A[i]와 A[least]의 교환;
    i++;
```

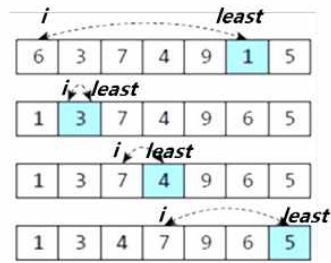


```
#define SWAP(x, y, t) ( (t)=(x), (x)=(y), (y)=(t) )
```

```
void selection_sort(int list[], int n)
```

```
{
    int i, j, least, temp;

    for(i = 0; i < n-1; i++) {
        least = i;
        for(j = i+1; j < n; j++)
            if(list[j] < list[least])
                least = j;
        SWAP(list[i], list[least], temp);
    }
}
```



정렬되지 않은 오른쪽 부분 배열
에서 최소값 인덱스를 탐색

```
#define MAX_SIZE 10
```

```
int list[MAX_SIZE];
```

```
int main(void)
```

```
{
    int i;
    n = MAX_SIZE;
    srand(time(NULL));
    for (i = 0; i < n; i++)          // 난수 생성 및 출력
        list[i] = rand() % 100;      // 난수 발생 범위 0~99

    selection_sort(list, n);          // 선택정렬 호출
    for (i = 0; i < n; i++)
        printf("%d ", list[i]);
    return 0;
}
```

출력 : 16 24 25 38 48 64 87 90 93 96

선택정렬 복잡도

시간복잡도 : $O(\quad)$ $= (n-1) + (n-2) + \dots + 1 = n(n-1)/2$

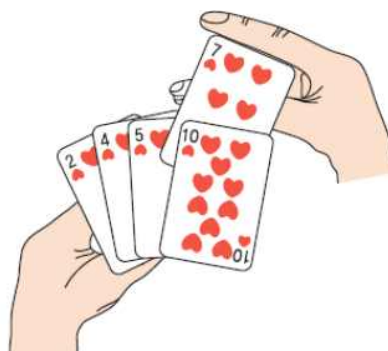
특징 : 알고리즘이 매우 간단, but 효율적이지는 않음

in-place 정렬

자료 이동 횟수가 미리 결정됨 worst - avg - best 모두 $O(n^2)$

2. 삽입 정렬 Insertion Sort

기존의 정렬된 부분에 새로운 레코드를 올바른 위치를 찾아 삽입하는 과정을 반복





```

insertion_sort(A, n)
for i ← 1 to n-1 do
    key ← A[i];
    j ← i-1;      // j는 i 바로 앞
    while j≥0 and A[j]>key do
        A[j+1] ← A[j];
        j ← j-1;
    A[j+1] ← key
  
```

실제 코드

```

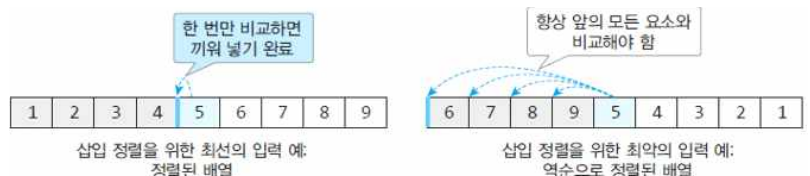
// 삽입정렬
void insertion_sort(int list[], int n)
{
    int i, j, key;
    for(i=1; i<n; i++){
        key = list[i];
        for(j=i-1; j>=0 && list[j]>key; j--){
            list[j+1] = list[j];      // 레코드의 오른쪽으로 쉬프트
        }
        list[j+1] = key;
    }
}
  
```



삽입정렬 복잡도

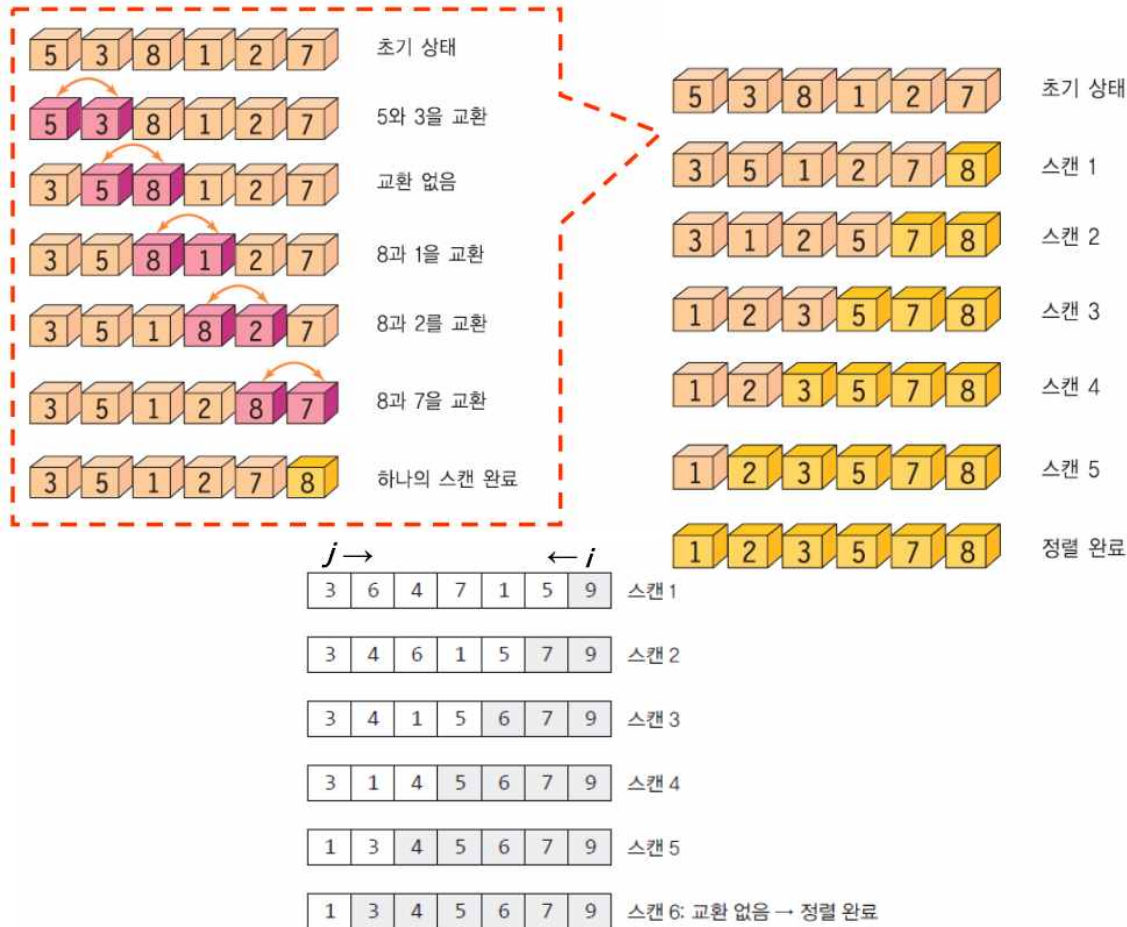
시간복잡도 : **best** $O(n)$ - 정렬이 미리 되어있으면 $O(n)$. 오름차순으로 정렬이 되어 있을 때
worst $O(n^2)$ - 내림차순으로 정렬이 되어 있을 때

특징 : 많은 이동이 필요. 레코드 크기가 크면 불리. 느려짐
 제자리 정렬
 레코드가 대부분 정렬이 되어 있다면 매우 효율적



3. 버블 정렬 Bubble sort

인접한 두개의 레코드를 비교하여 순서대로 되어 있지 않으면 서로 교환하는 방법



BubbleSort(A, n)

for $i \leftarrow n-1$ to 1 do

for $j \leftarrow 0$ to $i-1$ do

j 와 $j+1$ 번째의 요소가 크기 순이 아니면 서로 교환

$j++$;

$i--$;

실제 코드

```
#define SWAP(x, y, t) ( (t)=(x), (x)=(y), (y)=(t) )
```

```
void bubble_sort(int list[], int n)
```

```
{
```

```
int i, j, temp;
```

```
for(i=n-1; i>0; i--){
```

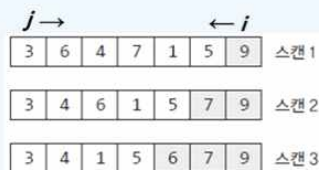
```
for(j=0; j<i; j++){
```

```
if(list[j]>list[j+1]) // 앞뒤의 레코드를 비교한 후 교체
```

```
SWAP(list[j], list[j+1], temp);
```

```
}
```

```
}
```



삽입정렬 복잡도

시간복잡도 : $O(n^2)$ = $n(n-1)/2$

특징 : 많은 이동이 필요. 레코드 크기가 크면 불리. 가장 왼쪽 요소-> 맨 오른쪽 가려면 교환 多
in place 정렬
알고리즘은 단순하지만, 이동연산의 오버헤드로 거의 사용되지 않음.

4. 병합 정렬 Merge Sort

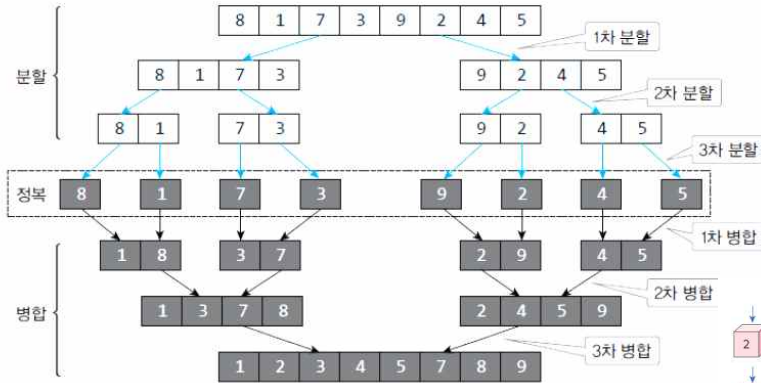
리스트를 두 개의 균등한 크기로 분할하고 분할된 부분 리스트를 정렬
정렬된 두 개의 부분 리스트를 합하여 전체 리스트를 정렬

Divide & Conquer 전략

Divide : 입력 배열을 같은 크기의 2개의 부분 배열로 분할

Conquer : 부분 배열을 정렬. 부분 배열의 크기가 충분히 작지 않으면 순환 호출을 이용해 다시 Divide & Conquer.

Combine : 정렬된 부분 배열을 하나의 배열에 병합



```
merge_sort(list, left, right)
```

```
if left < right
```

```
    mid = (left+right)/2;
    merge_sort(list, left, mid);
    merge_sort(list, mid+1, right);
    merge(list, left, mid, right);
```

```
merge(list, left, mid, right):
```

```
// 2개의 인접한 배열 list[left..mid]와 list[mid+1..right]를 합병
```

```
i ← left;
```

```
j ← mid+1;
```

```
k ← left;
```

```
while i ≤ mid and j ≤ right
```

```
    if(list[i] < list[j])
```

```
        sorted[k] ← list[i]; //sorted[]는 병합된 값들을 임시 저장하는 배열
        k++;
        i++;
```

```
    else
```

```
        sorted[k] ← list[j];
        k++;
        j++;
```

```
요소가 남아있는 부분배열을 sorted로 복사한다;
```

```
sorted를 list로 복사한다;
```

```
void merge_sort(int list[], int left, int right)
```

```
{
```

```
    int mid;
```

```
    if(left < right) {
```

```
        mid = (left+right)/2; // 리스트의 균등분할
```

```
        merge_sort(list, left, mid); // 부분리스트 정렬
```

```
        merge_sort(list, mid+1, right); // 부분리스트 정렬
```

```
        merge(list, left, mid, right); // 합병
```

```
    }
```

```
}
```




```
int sorted[MAX_SIZE]; // 추가 공간이 필요
// i는 정렬된 왼쪽리스트에 대한 인덱스
// j는 정렬된 오른쪽리스트에 대한 인덱스
// k는 정렬될 리스트에 대한 인덱스
```

```
void merge(int list[], int left, int mid, int right)
```

```
{
    int i, j, k, l;
    i=left; j=mid+1; k=left;

    // 분할 정렬된 list의 합병
    while(i<=mid && j<=right){
        if(list[i]<=list[j])
            sorted[k++] = list[i++];
        else
            sorted[k++] = list[j++];
    }

    if(i>mid) // 남아 있는 레코드의 일괄 복사
        for(l=j; l<=right; l++)
            sorted[k++] = list[l];
    else // 남아 있는 레코드의 일괄 복사
        for(l=i; l<=mid; l++)
            sorted[k++] = list[l];

    // 배열 sorted[]의 리스트를 배열 list[]로 복사
    for(l=left; l<=right; l++)
        list[l] = sorted[l];
}
```



병합정렬 복잡도

시간복잡도 :

$O(n \log n)$

크기가 n 인 리스트를 정확히 균등 분배하므로 $\log n$ 개의 단계 생성
각 단계당 레코드 n 개를 비교

특징 :

효율적임. 입력의 구성과 상관없이 동일한 시간에 정렬 완료
in-plane 정렬이 아님

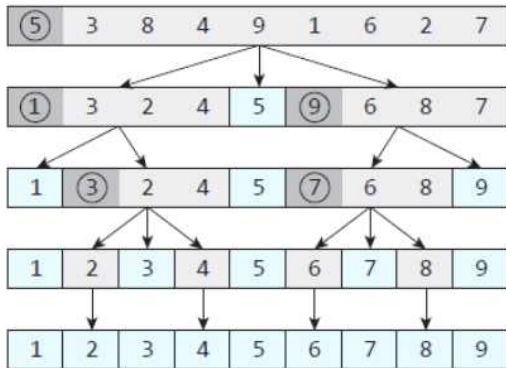
$n \log n$ 보장!

5. 퀵 정렬 Quick Sort

pivot값을 기준으로 리스트를 2개의 부분 리스트로 **비균등** 분할. 각 부분 리스트를 다시 재귀적으로 퀵정렬 수행 (divide & conquer)

평균적으로 가장 빠른 정렬 방법





[그림 9.16] 퀵 정렬의 전체 과정

알고리즘 9.6 퀵 정렬 알고리즘

```

01 quick_sort(A[], left, right)
02     if left < right :
03         q ← partition(A, left, right)
04         quick_sort(A, left, q-1)
05         quick_sort(A, q+1, right)

```

// 정렬할 요소가 2개 이상이면
 // 분할하고 피벗의 인덱스 반환
 // 왼쪽 부분 배열을 정렬
 // 오른쪽 부분 배열을 정렬

알고리즘 9.7 분할 알고리즘

```

01 partition(A[], left, right)
02     pivot ← A[left]
03     low ← left+1
04     high ← right
05     while low ≤ high :
06         while low ≤ high and A[low] ≤ pivot : low ← low+1
07         while low ≤ high and A[high] > pivot : high ← high-1
08         if low < high :
09             A[low] ↔ A[high]
10
11     A[left] ↔ A[high]
12     return high

```

// 피벗으로 첫 번째 요소 사용
 // 왼쪽 배열을 위한 인덱스
 // 오른쪽 배열을 위한 인덱스
 // 조건에 맞지 않는 두 요소
 // 교환
 // 피벗을 제 위치로 이동
 // 피벗의 위치를 반환

교
본

```
int partition(int list[], int left, int right)
```

```
{
```

```
    int pivot, temp;
```

```
    int low, high;
```

```
    low = left+1;
```

```
    high = right;
```

```
    pivot = list[left];
```

```
    while(low <= high){
```

```
        while(low <= right && list[low] <= pivot)
```

```
            low++;
```

```
        while(high >= left && list[high] > pivot)
```

```
            high--;
```

```
        if(low < high)
```

```
            SWAP(list[low], list[high], temp);
```

```
    };
```

```
    SWAP(list[left], list[high], temp);
```

```
    return high;
```

```
}
```

```
void quick_sort(int list[], int left, int right)
```

```
{
```

```
    if (left < right) {
```

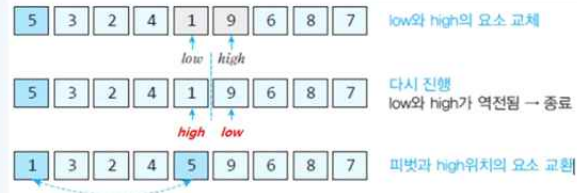
```
        int q = partition(list, left, right);
```

```
        quick_sort(list, left, q - 1);
```

```
        quick_sort(list, q + 1, right);
```

```
    }
```

```
}
```



퀵정렬 복잡도

시간복잡도 : **best** $O(n \log n)$ --- 균등하게 분할된 경우
 worst $O()$ --- 한쪽으로 치우쳐진 경우

>>Pivot을 어떤걸로 잡느냐가 관건!

특징 : 효율적인 알고리즘.
 불필요한 데이터 이동이 적음
 먼거리의 데이터를 교환
 한번 결정된 피벗(위치가 확정된 것들)은 추후 연산에서 제외

개선책 : **중간값(median)**을 피벗으로 선택하면 불균등 완화

Median of Three : 일반적으로 3개의 값(리스트 왼쪽, 오른쪽, 중간위치) 중에서 median 값을 선택

6. 기수 정렬 Radix Sort

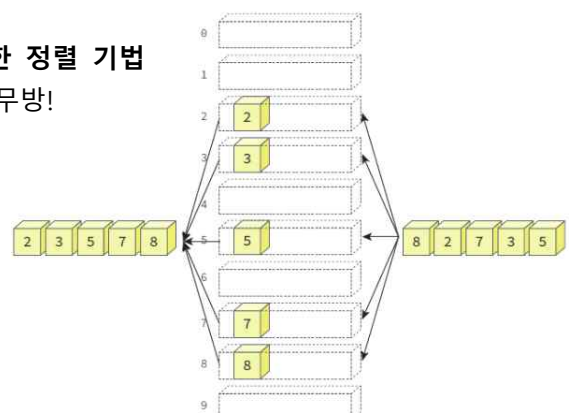
레코드끼리 **비교하지 않고** 정렬하는 독특한 기법

대부분 정렬 방법은 레코드를 **비교함**으로써 정렬을 수행. -> 비교기반 정렬은 $O(n \log n)$ 을 넘을 수 없음!

비교기반 정렬의 이론적인 시간 복잡도 **하한선을 깰 수 있는 유일한 정렬 기법**

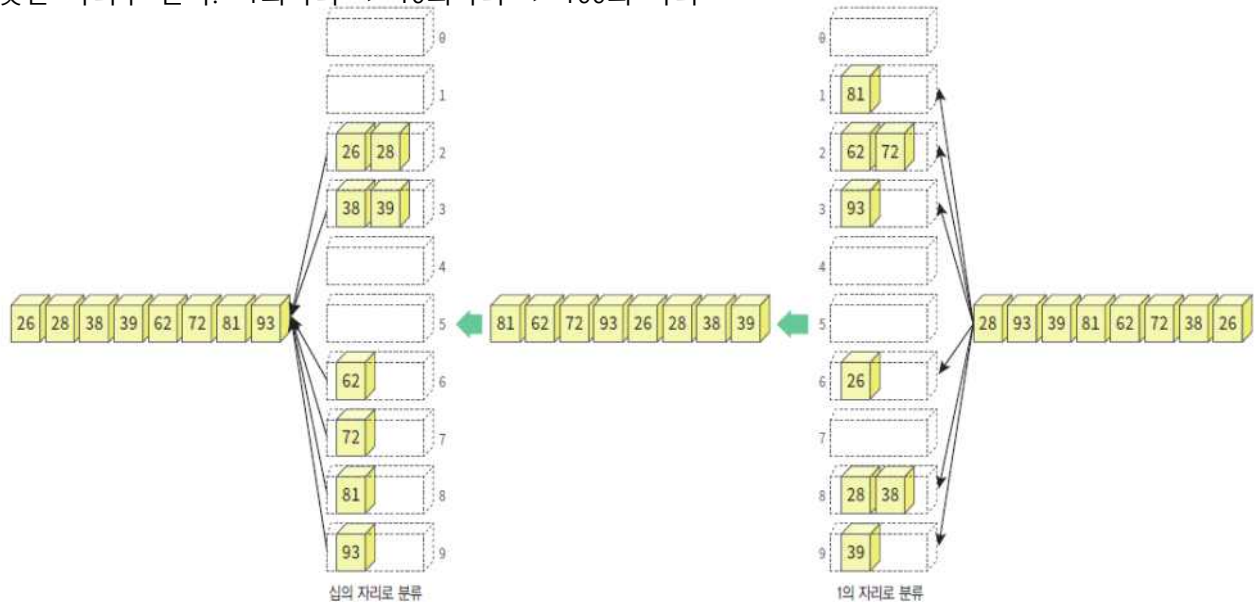
d는 데이터의 자릿수 : 거의 **선형 알고리즘 $O(n)$** 이라고 봐도 무방!

원리 : 데이터의 자릿수만큼 버킷(Bucket)을 사용
 단순히 해당 큐에 넣었다가 빼내면 됨



여러 자리수 정렬

낮은 자리수 먼저! 1의자리 -> 10의자리 -> 100의 자리



알고리즘 9.8 기수정렬 알고리즘

```

01 radix_sort(A[], n)
02     factor ← 1                                // 1의 자리부터 시작
03     for d←0 to DIGITS-1 :                      // 모든 자릿수에 대해
04         for i←0 to n-1 :                      // 배열의 모든 요소를
05             id = (A[i]/factor) % BUCKETS      // 해당 버킷을 찾아
06             Q[id].enqueue(A[i])              // 그 버킷에 삽입
07
08         i←0                                    // 버킷에서 꺼내는 작업 시작
09         for b←0 to BUCKETS-1 :                // 모든 버킷에서
10             while not Q[b].is_empty()         // 모든 요소를 순서대로 모두 꺼내
11                 A[i] ← Q[b].dequeue()         // 입력 배열에 다시 저장
12                 i ← i + 1
13
14         factor ← factor * BUCKETS              // 다음 자릿수로 올라감

#define BUCKETS 10
#define DIGITS 4
void radix_sort(int list[], int n)
{
    int i, b, d, factor=1;
    QType queues[BUCKETS];                      // 큐 관련 코드는 6장 큐 구현 부분 참조

    for(b=0;b<BUCKETS;b++) init(&queues[b]); // 큐들의 초기화
    for(d=0; d<DIGITS; d++){
        for(i=0;i<n;i++)                      // 데이터들을 자릿수에 따라 각 큐에 입력
            enqueue( &queues[(list[i]/factor)% BUCKETS], list[i]);
        i = 0;
        for(b=0;b<BUCKETS;b++) // 각 큐에서 꺼내어 list로 이동
            while( !is_empty(&queues[b]) )
                list[i++] = dequeue(&queues[b]);
        factor *= 10;                          // 그 다음 자릿수로 이동
    }
}

```

Radix sort 복잡도

시간복잡도 : $O(dn)$ d 는 digit 자릿수 n 은 레코드의 수

한계점 : 정렬할 수 있는 레코드 타입이 제한적

키가 동일한 길이를 가지는 숫자나 단순문자여야 함

실수, 한글, 한자 등으로 구성된 키는 정렬 불가

알고리즘	최선	평균	최악
선택 정렬	$O(n^2)$	$O(n^2)$	$O(n^2)$
삽입 정렬	$O(n)$	$O(n^2)$	$O(n^2)$
버블 정렬	$O(n)$	$O(n^2)$	$O(n^2)$
퀵 정렬	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n^2)$
병합 정렬	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n \log_2 n)$
기수 정렬	$O(dn)$	$O(dn)$	$O(dn)$

□ 실험 예: 정수 60,000개 정렬 시간 (초)

알고리즘	실행시간 (초)
삽입정렬	7.438
선택정렬	10.842
버블정렬	22.894
병합정렬	0.026
퀵정렬	0.014

□ Bubble 정렬의 전체 시간 복잡도는 항상 $O(n^2)$
 Q: 이미 정렬이 되어 있다면 $O(n)$ 이 되도록 개선하시오

Lab: 버블 정렬 개선

```
#define SWAP(x, y, t) ( (t)=(x), (x)=(y), (y)=(t) )

void bubble_sort(int list[], int n)
{
    int i, j, temp;
    for(i=n-1; i>0; i--){
        int exchange_flag = 0;
        for(j=0; j<i; j++) // 앞뒤의 레코드를 비교한 후 교환
            if(list[j]>list[j+1]){
                SWAP(list[j], list[j+1], temp);
                exchange_flag = 1;
            }
        if(!exchange_flag) break; // 교환이 한 번도 없었으면 종료
    }
}
```

$j \rightarrow$

1	2	3	4	5	6	7
---	---	---	---	---	---	---

 $\leftarrow i$

최선의 경우 버블 정렬 복잡도: $O(n)$ 으로 개선

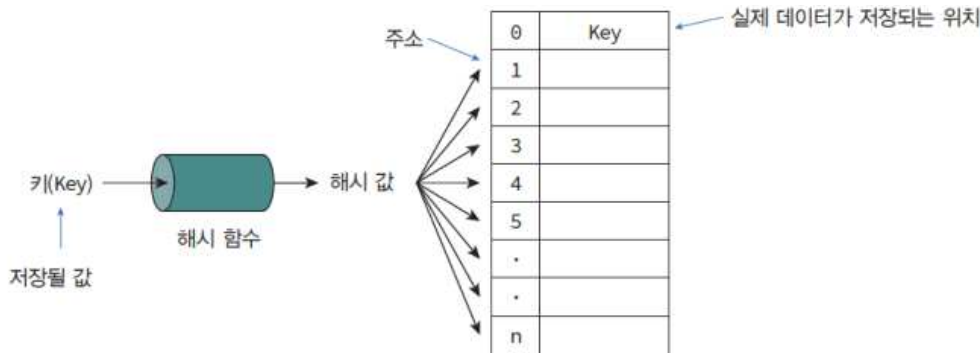
12장 해싱 Hashing

해싱(Hashing)이란?

기존 탐색 방법은 : 최대 가능한 시간 복잡도가 $O(\log n)$ binary search tree 등..

해싱(hasing) : 키 값에 산술적인 연산을 적용하여 항목이 저장되어 있는 테이블의 주소를 계산하여 항목을 곧바로 탐색하는 방법 -> 시간 복잡도 : $O(1)$

해시 테이블(hash table) : 키 값의 연산에 의해 직접 접근이 가능한 자료 구조



사전 Dictionary 구조

맵 또는 테이블로 불리움

탐색 키와 관련된 값의 2가지 필드로 구성

영어 단어나 사람의 이름 같은 탐색 키(search key)

단어의 정의나 주소 또는 전화 번호 같은 탐색 키와 관련된 값(value)



사전 구조 ADT

객체 : key : value 집합

- 연산 :
- **add(key, value) ::= (key,value)를 사전에 추가한다.**
 - **delete(key) ::= key에 해당되는 (key,value)를 찾아서 삭제한다.**
관련된 value를 반환한다. 만약 탐색이 실패하면 NULL를 반환한다.
 - **search(key) ::= key에 해당되는 value를 찾아서 반환한다.**
만약 탐색이 실패하면 NULL를 반환한다.

해싱의 구조

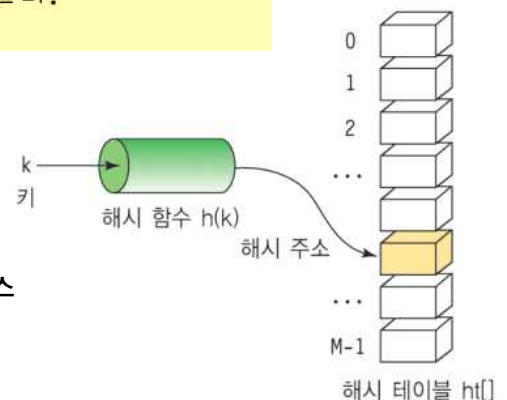
해싱은 해시 함수와 해시 테이블로 구성

해시 함수(hash function)

탐색키를 입력받아 해시 값(hash value) 생성

해시 값은 배열로 구현된 해시 테이블(hash table)의 인덱스

해시 값 == 해시 주소(hash address)



해시 테이블

M개의 버킷(bucket)으로 구성된 테이블

ht[0], ht[1], ..., ht[M-1]의 원소를 가짐.

하나의 버킷에 s개 슬롯(slot) 가능

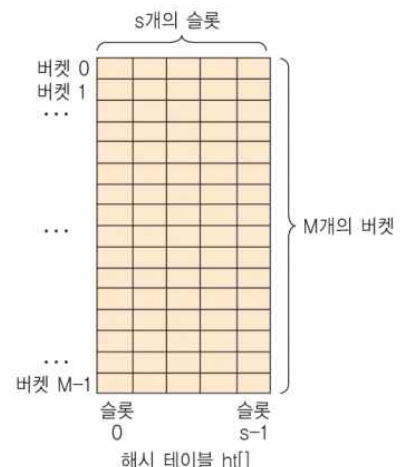
충돌 Collision

서로 다른 두 개의 탐색키 k1과 k2에 대해서 $h(k1) = h(k2)$ 인 경우

오버플로우 Overflow

충돌된 버킷에 할당된 슬롯 수보다 많이 발생하는 것

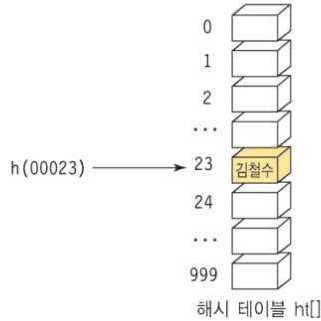
>> 해싱에서는 충돌과 오버플로우가 일어나고, 해결할 방법이 반드시 필요하다.



이상적인 해싱

학생 정보를 해싱으로 저장, 탐색하는 경우

- 학번 5자리 중 앞 2자리가 학과 번호, 뒤 3자리가 각 학과의 학생 번호라고 가정
- 같은 학과 학생들만 가정하면 뒤의 3자리만 사용해서 탐색 가능
- 학번이 00023이라면 이 학생의 인적사항은 해시테이블 ht[23]에 저장
- 만약 해시테이블이 1000개의 공간을 가지고 있다면 탐색 시간이 $O(1)$ 이 되므로 이상적임



7

>> 버킷 갯수가 많으면 충돌 확률 감소.

실제 해싱

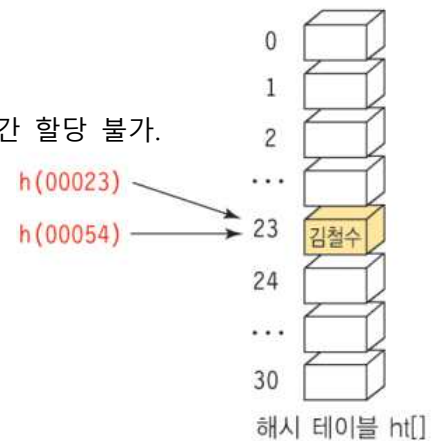
실제로 해시테이블의 크기는 제한되므로, 존재 가능한 모든 키에 대해 저장공간 할당 불가.
필연적으로 충돌과 오버플로우 발생

해시함수 : $h(k) = k \bmod M$

알파벳 문자열 키의 해시함수가 키의 첫번째 문자의 순서라고 가정.

$h(\text{"array"}) = 1$

$h(\text{"binary"}) = 2$

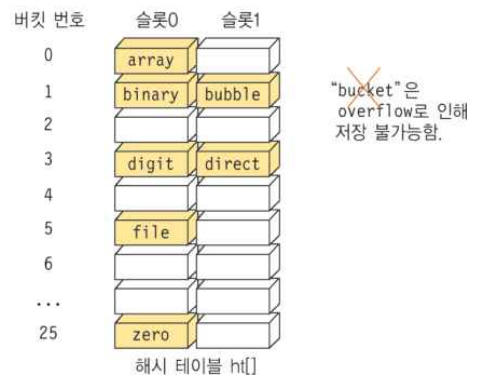


입력데이터: array, binary, bubble, file, digit, direct, zero, bucket

해시 함수

좋은 해시 함수의 조건

1. 충돌이 적어야 함.
2. 해시함수 값이 해시테이블 주소 영역 내에서 고르게 분포해야 함
3. 계산이 빨라야 함.



제산 함수 (Division)

$h(k) = k \% M$ k 는 키값 M 은 테이블 크기(주로 prime number)

가장 일반적인 해시 함수

접지 함수 (Folding)

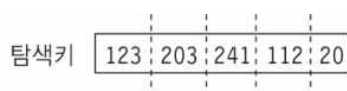
키를 몇 개 부분으로 나누어서 이를 더하거나 비트별 XOR을 수행

키의 크기가 해시 테이블 크기보다 더 큰 정수일 경우 주로 사용

이동 폴딩(Shift Folding)과 경계 폴딩(Bounding Folding)

Shift Folding : 여러 부분으로 나눈 키 값을 더해서 해시 주소로 사용

Boundary Folding : 여러 부분으로 나눈 키값들 중 이웃 키 부분을 거꾸로 더 해서 해시 주소로 사용



$$\text{이동폴딩} \quad 123 + 203 + 241 + 112 + 20 = 699$$

$$\text{경계폴딩} \quad 123 + 302 + 241 + 211 + 20 = 897$$

중간 제곱(Mid Square)

키를 제곱한 다음, **중간의 몇 비트를 취해서** 해시 주소 생성

해시 테이블 크기(M)이 일 때 주로 사용

키를 **이진수**로 간주하여 임의의 위치(주로 중간)의 **k개의 비트**를 해시 주소로 사용

테이블 크기 = 256 = 2^8 이라고 가정하면 256개의 값을 만들기 위해서 8비트가 필요.

키 = 13735일 경우, 이 수를 **제공**한다. $13735^2 = 188650225$

= 000010110011**11101001**00101011110001

가운데 8비트 11101001 = 223을 해시 주소로 사용

숫자 분석 함수 (Digit analysis)

키의 각 자릿수 분포를 미리 분석해서 편중되지 않은 수를 해시 주소로 사용

학번이 202512345라고 할 경우 입학년도를 의미하는 앞의 4자리는 편중 이 심하므로 가급적 사용하지 않고 나머지 수를 조합하여 사용

충돌(Collision)

서로 다른 탐색 키를 갖는 항목들이 **같은 해시 주소**를 가지는 현상

충돌이 발생하면 해시 테이블에 항목 **저장 불가능**

충돌을 효과적으로 해결하는 방법이 반드시 필요함.

충돌 해결책

개방 주소방식(Open addressing) : 충돌 발생시 해시 테이블 내 **다른 위치**를 찾아 데이터 저장

선형조사(linear probing), 이차조사(quadratic probing), 이중해싱 (double hashing)

폐쇄 주소방식(Closed addressing) : 충돌 발생시 같은 해시주소에 해당하는 모든 값들은 하나의 리스트에 차례로 저장

체이닝(chaining)

선형조사법 Linear Probing

충돌이 $ht[k]$ 에서 발생했다면

>> $ht[k+1]$ 이 비어있는지 체크

>> 아니라면 $ht[k+2]$ 조사

>> 비어있는데가 나올 때까지 계속 조사. 테이블의 끝에 도달하게 되면 다시 처음부터 조사

조사를 시작했던 곳으로 다시 되돌아오게 되면 테이블이 가득 찬 것임.

조사 위치: $h(k)$, $h(k)+1$, $h(k)+2$,...

Clustering 현상이 제일 심함!

□ (예) $h(k) = k \bmod 7$

1 단계 (8) : $h(8) = 8 \bmod 7 = 1$ (저장)
2 단계 (1) : $h(1) = 1 \bmod 7 = 1$ (충돌 발생)
 $(h(1)+1) \bmod 7 = 2$ (저장)
3 단계 (9) : $h(9) = 9 \bmod 7 = 2$ (충돌 발생)
 $(h(9)+1) \bmod 7 = 3$ (저장)
4 단계 (6) : $h(6) = 6 \bmod 7 = 6$ (저장)
5 단계 (13) : $h(13) = 13 \bmod 7 = 6$ (충돌 발생)
 $(h(13)+1) \bmod 7 = 0$ (저장)

	1단계	2단계	3단계	4단계	5단계
[0]					13
[1]	8	8	8	8	8
[2]		1	1	1	1
[3]			9	9	9
[4]					
[5]					
[6]				6	6

이차조사법 Quadratic Probing

선형조사와 유사하지만, 다음 조사할 위치를 아래 식 사용.

$$(h(k) + i^2) \bmod M, \text{ where } i = 0, 1, 2, \dots, M-1$$

조사위치 : $h(k), h(k)+1, h(k)+4, h(k)+9, \dots$

이차조사는 선형조사보다 Clustering 문제를 완화.

>> 하지만, 여전히 충돌이 일어난 키들이 특정패턴으로 모일 수 있음(2차 클러스터링)

우선 $(7) \bmod 10 = 3$
 없으면 >> $(7+1) \bmod 10 = 8$
 없으면 >> $(7+2^2) \bmod 10 = 1$
 없으면 >> $(7+3^2) \bmod 10 = 6$
 없으면 >> $(7+4^2) \bmod 10 = 3$
 없으면 >> $(7+5^2) \bmod 10 = 2$

이중해싱 Double Hasing

rehashing(재해싱)이라고도 함.

다음 조사할 위치 결정 시 원래 해시함수와는 다른 별개의 해시함수를 사용

두 번째 해시함수는 **조사 간격을 결정**

$$h'(k) = C - (k \bmod C), \quad \text{where } 1 \leq h'(k) \leq C$$

조사 위치 : $h(k), h(k)+h'(k), h(k)+2h'(k), h(k)+3h'(k), \dots$

클러스터링 문제를 매우 효율적으로 해결

해시함수 값이 같더라도 키다 다르면 서로 조사 순서가 다른
 추가적인 해시함수 계산 **오버헤드** 발생

예) 해시테이블 크기가 7이고 입력 (8,1,9,6,13) 가정,

- ▣ 첫 번째 해시 함수: $h(k) = k \bmod 7$
- ▣ 두 번째 해시 함수: $h'(k) = 5 - (k \bmod 5)$

1단계 (8) : $h(8) = 8 \bmod 7 = 1$ (저장)
 2단계 (1) : $h(1) = 1 \bmod 7 = 1$ (충돌발생)
 $(h(1)+h'(1)) \bmod 7 = (1+5-(1 \bmod 5)) \bmod 7 = 5$ (저장)
 3단계 (9) : $h(9) = 9 \bmod 7 = 2$ (저장)
 4단계 (6) : $h(6) = 6 \bmod 7 = 6$ (저장)
 5단계 (13) : $h(13) = 13 \bmod 7 = 6$ (충돌 발생)
 $(h(13)+h'(13)) \bmod 7 = (6+5-(13 \bmod 5)) \bmod 7 = 1$ (충돌발생)
 $(h(13)+2 \cdot h'(13)) \bmod 7 = (6+2 \cdot 2) \bmod 7 = 3$ (저장)

	1단계	2단계	3단계	4단계	5단계
[0]					
[1]	8	8	8	8	8
[2]			9	9	9
[3]					13
[4]					
[5]		1	1	1	1
[6]				6	6

체이닝 Chaining

해시 충돌 문제를 linked list로 해결

각 버킷에 삽입과 삭제가 용이한 **연결 리스트** 할당

버킷 내에서는 연결 리스트 순차 탐색

장점 : 개방 주소방식(Open Addressing)에서 발생하는 클러스터링 문제가 없음

해시테이블 크기에 덜 민감. 해시테이블 채워질수록 성능저하가 linear하게(선형적으로) 발생

개방주소방식에 비해 **저장할 데이터가 많을 때 더 유리**

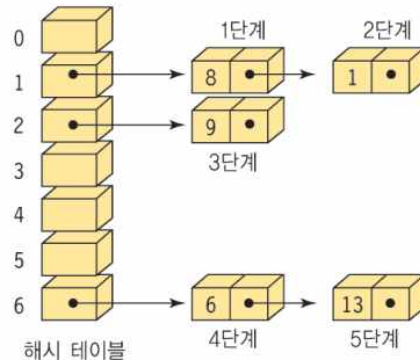
단점 : 메모리 사용량 증가(리스트 노드들의 오버헤드)

데이터 삽입/삭제 시 오버헤드가 더 큼

□ 예) 해시테이블 크기가 7이고 입력 (8,1,9,6,13) 가정,

□ 해시 함수: $h(k) = k \bmod 7$

1단계 (8) : $h(8) = 8 \bmod 7 = 1$ (저장)
2단계 (1) : $h(1) = 1 \bmod 7 = 1$ (충돌발생->새로운 노드 생성 저장)
3단계 (9) : $h(9) = 9 \bmod 7 = 2$ (저장)
4단계 (6) : $h(6) = 6 \bmod 7 = 6$ (저장)
5단계 (13) : $h(13) = 13 \bmod 7 = 6$ (충돌 발생->새로운 노드 생성 저장)



```
#define TABLE_SIZE    7           // 해시 테이블의 크기=소수
                                // 해시 테이블을 Prime number로 하는 이유는 물리지 않게 하기 위함.
                                // 보조 해시와의 궁합때문에 Prime number로 하는 것이 좋음!

typedef struct {
    int key;
} element;

typedef struct list {
    element item;
    struct list *link;
}ListNode;

ListNode *hash_table[TABLE_SIZE]; // 해시 테이블

// 제산 함수를 사용한 해시 함수
int hash_function(int key)
{
    return key % TABLE_SIZE;
}
```

```

void hash_chain_add(element item, ListNode *ht[])
{
    int hash_value = hash_function(item.key);
    ListNode *ptr;
    ListNode *node_before = NULL, *node = ht[hash_value];
    for (; node; node_before = node, node = node->link) {
        if (node->item.key == item.key) {
            printf("이미 탐색키가 저장되어 있음\n");
            return;
        }
    }
    ptr = (ListNode *)malloc(sizeof(ListNode));
    ptr->item = item;
    ptr->link = NULL;
    if (node_before)
        node_before->link = ptr;
    else
        ht[hash_value] = ptr;
}

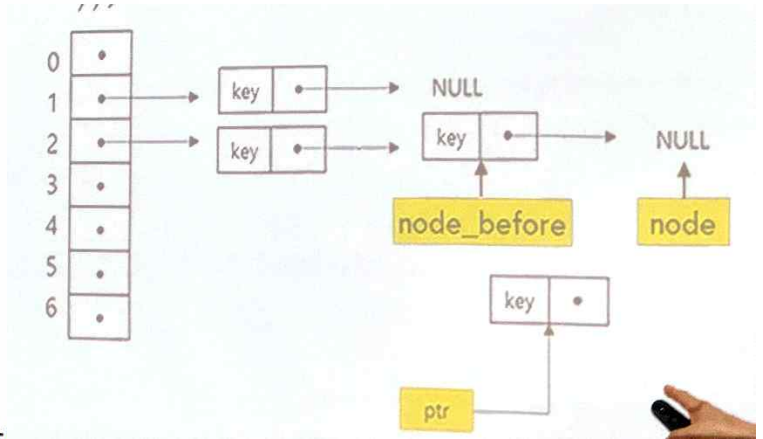
```

```

// 체인법을 이용하여 테이블에 저장된 키를 탐색
void hash_chain_search(element item, ListNode *ht[])
{
    ListNode *node;

    int hash_value = hash_function(item.key);
    for (node = ht[hash_value]; node; node = node->link) {
        if (node->item.key == item.key) {
            printf("탐색 %d 성공 \n", item.key);
            return;
        }
    }
    printf("키를 찾지 못했음\n");
}

```



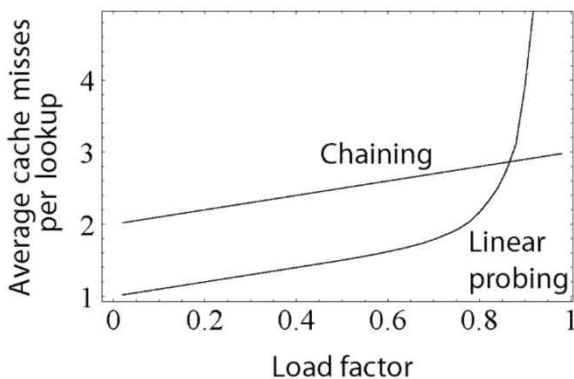
적재 비율(Load Factor)

저장되는 항목의 개수 n 과 해시 테이블 크기 M 의 비율
체인닝에서는 버킷당 평균 연결 리스트의 길이

가 1을 넘을수도 있다!

$$\alpha = \frac{\text{저장된 항목의 개수}}{\text{해싱 테이블의 버킷의 개수}} = \frac{n}{M}$$

해싱의 성능 분석



선형조사

α	실패한 탐색	성공한 탐색
0.1	1.1	1.1
0.3	1.5	1.2
0.5	2.5	1.5
0.7	6.1	2.2
0.9	50.5	5.5

체인닝

α	실패한 탐색	성공한 탐색
0.1	0.1	1.1
0.3	0.3	1.2
0.5	0.5	1.3
0.7	0.7	1.4
0.9	0.9	1.5
1.3	1.3	1.7
1.5	1.5	1.8
2.0	2.0	2.0

1. 체이닝 : 데이터가 많을 때 유리

2. 선형조사 : 해시테이블 load factor를 절반이하로 유지해야 함.